

SUMMARY: VECTOR DATABASES-II

Objective

By the end of this session, students will be able to:

1. Explain the differences between dense, sparse, and hybrid indexes, and when to use each in a retrieval pipeline.
2. Describe how quantization works and its role in scaling vector search without significant accuracy loss.
3. Understand the role of rerankers in improving retrieval quality and how they integrate into multi-stage search pipelines.
4. Discuss latency optimization techniques, including caching and batch similarity searches, and how Pinecone automates parts of this process.
5. Compare Pinecone and FAISS in terms of query caching, batching, and scalability.

Introduction to types of Indexes:

Up to this point, we've been working with **dense indexes** in FAISS — like **HNSW** and **IVF** — where:

- Every document is converted into a **dense vector** using a model such as all-MiniLM-L6-v2.
- Queries are also converted into vectors using the **same model**.
- Retrieval is done by **similarity matching** (cosine similarity or inner product) between the query vector and document vectors.
- The index (HNSW, IVF) is optimized to quickly find the most similar vectors without scanning them all.

This approach excels at **semantic search** — finding documents that are related in meaning even if they don't share the exact same keywords.

1. Dense Indexes: Capturing Semantic Meaning

Dense indexes store dense vectors—numerical representations where every dimension contains meaningful information about the semantic relationships within data. These vectors typically range from 100-1000 dimensions and are generated by machine learning models like BERT, OpenAI's text-embedding models, or Sentence Transformers.

Key characteristics of dense vectors:

- Every dimension contains a non-zero value representing learned features
- Capture semantic meaning and contextual relationships
- Excel at understanding synonyms, paraphrases, and conceptual similarity
- Generated by neural networks trained on large text corpora

Dense indexes enable semantic search, where queries like "buying a car" can match documents containing "purchasing an automobile" even without exact keyword overlap

Let's first explore some of the popular dense vectors models:

- **OpenAI Embeddings** – e.g., `"text-embedding-3-small"` or `"text-embedding-3-large"` from OpenAI API.
- **Google Gemini Embeddings** – via `models/embedding-001` from Google Generative AI API.
- **Pinecone Serverless Dense Embeddings** – Pinecone's own hosted dense vector services.
- **Sentence Transformers** – e.g., `"all-MiniLM-L6-v2"`, `"multi-qa-mpnet-base-dot-v1"`.
- **Cohere Embeddings** – e.g., `"embed-english-v3.0"` or `"embed-multilingual-v3.0"`.
- **Hugging Face Transformers** – e.g., `"bert-base-uncased"`, `"distilroberta-base"`.
- **Anthropic Claude Embeddings** (beta/private) – upcoming dense embedding model variants.
- **Nomic Embeddings** – e.g., `"nomic-embed-text-v1.5"` from Nomic AI.

Then let's talk of some examples of dense indexes (most we have already seen):

- **IndexFlatL2** – Exact search using Euclidean distance (L2).

- **IndexFlatIP** – Exact search using inner product (cosine similarity when vectors are normalized).
- **IndexHNSWFlat** – Graph-based Approximate Nearest Neighbor (ANN) search using HNSW over raw vectors.
- **IndexIVFFlat** – Inverted File (IVF) index for approximate search over raw vectors.
- **IndexIVFPQ** – IVF with Product Quantization for large-scale memory-efficient search.
- **IndexIVFSQ8** – IVF with Scalar Quantization (8-bit) to reduce memory usage while keeping speed.
- **IndexIVFScalarQuantizer** – IVF with configurable scalar quantizers (like SQ4, SQ8).

Here's a table you can use to explain **dense index types in FAISS**:

Index Type	Distance Metric	Exact / Approximate	Key Idea	Best Use Case
IndexFlatL2	L2 (Euclidean)	Exact	Stores all vectors, brute-force search	Small datasets where accuracy is critical
IndexFlatIP	Inner Product (Cosine if normalized)	Exact	Brute-force search using inner product	Small datasets, semantic similarity with normalized vectors
IndexHNSWFlat	L2 / IP	Approximate	Graph-based ANN search (HNSW)	Medium-large datasets where speed is important
IndexIVFFlat	L2 / IP	Approximate	IVF clustering, search only relevant clusters	Large datasets where memory is not an issue
IndexIVFPQ	L2 / IP	Approximate	IVF + Product Quantization for compression	Very large datasets with limited RAM

IndexIVFSQ8	L2 / IP	Approximate	IVF + 8-bit scalar quantization	Large datasets needing balance between speed and memory
IndexIVFScalarQuantizer	L2 / IP	Approximate	IVF + configurable scalar quantization (e.g., SQ4, SQ8)	Memory-constrained environments with large datasets

2. Concept of Vector Quantization

So far, we've been talking about **dense indexes** in FAISS — IndexFlat, IVF, HNSW, etc.

Dense indexes store **high-dimensional vectors** (like 768 numbers for OpenAI embeddings or 384 numbers for Sentence Transformers) exactly as they are.

Now, here's the problem:

If you have **100 million vectors**, each 768 floats, that's:

```
100,000,000 × 768 × 4 bytes ≈ 300 GB
```

That's huge — even before indexing overhead.

The more vectors you store, the more memory you need. And when the index is **too big for RAM**, retrieval becomes slow or impossible.

That's where **quantization** comes in.

It's like compressing your vectors without losing too much of the meaning.

The Core Idea of Vector Quantization

Mathematically, quantization is about **approximating a continuous set of values with a limited set of representative values**.

Think of your vectors as points in a very high-dimensional space.

Each coordinate is a floating-point number — say **0.2345678**.

instead of storing that exact value, we store something *close enough* from a smaller set of possible numbers.

This smaller set is called a **codebook**.

2.1 How it works

Step 1 – Break the Vector Space into Chunks

Imagine your vector V is:

```
V = [0.91, 0.43, 0.05, -0.34, 0.88, ...] (dimension = 768)
```

We split this vector into smaller sub-vectors — for example, 8 numbers at a time.

So now you have 96 small vectors instead of 1 big one.

Step 2 – Create a Codebook for Each Chunk

For each chunk, we run k-means clustering to find, say, 256 cluster (0-255 since $2^8 = 256$) centers.

These centers are like “representative patterns” for that subspace.

Instead of storing the 8 real floats, we just store the index of the closest center (0 to 255).

That means instead of 8×4 bytes = 32 bytes, we store just 1 byte.

Step 3 – Store Codes Instead of Full Floats

After quantization, your 768-dimensional float vector becomes something like:

```
[ 23, 118, 5, 201, ... ] # each number is just 1 byte
```

That’s a **compression ratio of 32x** or more.

2.2 Why It Works for ANN Search

When FAISS searches, it doesn’t need the exact vector for all candidates.

It uses the compressed representation to narrow down potential matches, then (optionally) re-checks with the original vectors for the final top-k results.

You trade a **little bit of accuracy** for a **huge gain in memory efficiency and speed**.

Analogy to Remember

Think of your dense vector like a **high-resolution photograph** (exact floats).

Quantization is like creating a **color palette** of 256 colors and storing only which color each pixel uses.

The photo looks almost the same, but it's way smaller in size.

In FAISS terms, those 256 colors = the codebook.

```
import faiss
import numpy as np

# emb: (N, d) float32 array of your document vectors
# For cosine similarity, L2-normalize emb and use IP metric; for L2
# use as-is.

N, d = emb.shape
m = 16          # number of sub-quantizers (d should be divisible by m)
nbits = 8       # bits per sub-quantizer (K=256 centroids per subspace)

# For cosine similarity: normalize and use IP; here we show L2 path
# for clarity.
index_pq = faiss.IndexPQ(d, m, nbits)          # L2 by default
# If you want IP (cosine on normalized vectors):
# faiss.normalize_L2(emb)
# index_pq = faiss.IndexPQ(d, m, nbits, faiss.METRIC_INNER_PRODUCT)

# Train on a sample (or the whole set if small)
train_size = min(50000, N)
faiss_idx = np.random.choice(N, train_size, replace=False)
index_pq.train(emb[faiss_idx])

# Add all vectors (compressed as PQ codes)
index_pq.add(emb)

# Query
q = emb[0:1]  # or your own query embedding shaped (1, d)
D, I = index_pq.search(q, k=5)
print("Top-5 IDs:", I[0], "Approx distances:", D[0])
```

IVF + PQ:

```
import faiss
import numpy as np

N, d = emb.shape
m = 16          # PQ sub-quantizers (d % m == 0)
nbits = 8       # bits per sub-quantizer (256 centroids)
nlist = 4096     # IVF clusters (tune by dataset size; ~sqrt(N) is a
common rule)
nprobe = 16     # clusters to probe at query time (speed/recall knob)

# L2 path shown; for cosine: normalize emb and use IP
quantizer/metric
quantizer = faiss.IndexFlatL2(d)          # coarse
quantizer
index_ivfpq = faiss.IndexIVFPQ(quantizer, d, nlist, m, nbits) # L2
metric

# Train IVF coarse centroids + PQ codebooks
train_size = min(100000, N)
faiss_idx = np.random.choice(N, train_size, replace=False)
index_ivfpq.train(emb[faiss_idx])

# Add all vectors (stored as PQ codes inside IVF lists)
index_ivfpq.add(emb)

# Set query-time breadth
index_ivfpq.nprobe = nprobe

# Query
q = emb[0:1] # or your own query embedding shaped (1, d)
D, I = index_ivfpq.search(q, k=5)
print("Top-5 IDs:", I[0], "Approx distances:", D[0])
```

2.3 Disadvantages of Vector Quantization

- **Accuracy loss (quantization error):**

Distances are approximate; nearest neighbors can change vs exact search. You can mitigate by:

- increasing m (more subspaces) or nbits (larger sub-codebooks),
- **probing more IVF lists ($nprobe$)** if using IVF+PQ,
- re-ranking a small candidate set with exact vectors (if you can afford storing them or fetching them elsewhere).

- **Parameter sensitivity:**

M (used in Product Quantization), nbits, $nlist$, $nprobe$ need tuning per dataset. Poor choices = poor recall or wasted memory.

- **Training cost & distribution drift:**

Building codebooks requires k-means; it's non-trivial for very large data. If the data distribution shifts a lot later, codebooks can become suboptimal (you might need to retrain).

- **Independence assumption between subspaces:**

PQ assumes subspaces are independent; strong cross-dimension correlations can hurt. Mitigation: **OPQ (Optimized PQ)** learns a rotation before PQ to spread information more evenly across subspaces.

- **Limited arithmetic precision:**

With very small codebooks (e.g., 4 bits $\rightarrow$ 16 centroids), compression is great but approximation error grows.

- **Extra engineering for filtering/reranking:**

PQ indexes store compressed codes. If you need metadata filtering or exact re-ranking, you'll keep sidecar metadata and possibly the original vectors for a final exact distance pass.

Product Quantization (PQ)

- Directly building one huge codebook for all dimensions is expensive.
- So PQ **splits the vector into M parts** (sub-vectors), and each part is quantized separately.
- Each sub-vector has its own smaller codebook.

3. Sparse Indexes: Precision Through Exact Matching

Sparse indexes store sparse vectors—high-dimensional representations where most values are zero, with only a few non-zero values indicating specific term relevance. The most common sparse methods include BM25 and learned sparse models like SPLADE.

BM25 (Best Match 25) is one of the most widely used ranking algorithms in **information retrieval** systems (like search engines). It helps rank documents by their relevance to a query. You can think of it as an advanced version of **TF-IDF** (Term Frequency–Inverse Document Frequency).

A good example of sparse vectors is TF-IDF which we have already studied.

Key characteristics of sparse vectors:

- Majority of dimensions are zero (typically 99%+ sparsity)
- Stored efficiently as key-value pairs showing only non-zero positions
- Enable exact keyword matching and term-specific retrieval
- Dimension count often matches vocabulary size (e.g., 30,522 for BERT-based models)

Sparse indexes excel at lexical search, particularly valuable for domain-specific terminology, proper nouns, stock tickers, or cases requiring exact term matching

3.1 Sparse Index Architecture

Sparse indexes leverage inverted index data structures that map each vocabulary term to documents containing it. Pinecone's sparse implementation uses the MaxScore algorithm for efficient retrieval.

1. Query terms are mapped to their vocabulary positions
2. Inverted lists identify documents containing query terms
3. MaxScore algorithm scores documents in optimized order
4. Documents with guaranteed low relevance are pruned early
5. Top results are returned based on dot product scores

You can't really build a *native* sparse (BM25/TF-IDF inverted) index in FAISS—FAISS is built for **dense** (and binary) ANN. But you can wrap sparse text signals into FAISS in two simple ways:

1. **TF-IDF** → **Random hyperplane hashing** → FAISS **IndexLSH** (binary ANN)
2. **TF-IDF** → **TruncatedSVD (LSI)** → FAISS **dense index** (e.g., **IndexFlatIP**)

Below is a compact script that shows **both**. Option A gives you a “sparse-ish” FAISS pipeline via **binary hashing**; Option B shows the common **sparse**→**dense** reduction.

3.2 Building Sparse Index

Building a **true sparse (BM25/TF-IDF) index “inside FAISS” isn’t practical**. FAISS is designed for **dense (and binary) ANN**, not for inverted files, tokenization, or query parsing.

Why not sparse in FAISS?

- **No inverted index**: FAISS doesn’t store postings lists or term → doc mappings (the core of sparse IR).
- **No text pipeline**: No tokenization, stemming, stopwording, phrase/boolean queries, fields, analyzers.
- **No BM25/TF-IDF**: FAISS doesn’t implement lexical scoring or document length normalization.
- **Update & filtering pain**: Sparse IR needs efficient doc adds/deletes, field filters, and ranking—FAISS doesn’t provide those IR primitives.

```
# Minimal Whoosh BM25 over HTML docs
# pip install whoosh beautifulsoup4

from pathlib import Path
from datetime import datetime
from bs4 import BeautifulSoup
from whoosh import index
from whoosh.fields import Schema, ID, TEXT, NUMERIC
from whoosh.analysis import StemmingAnalyzer
from whoosh.qparser import QueryParser, MultifieldParser, OrGroup,
AndGroup, PhrasePlugin, FuzzyTermPlugin
from whoosh.query import TermRange
import tempfile, shutil

# --- Load HTML -> (doc_id, file_name, content, created_at) ---
def load_docs(html_dir: Path):
    out = []
    for fp in sorted(html_dir.glob("*.html")):
```

```
        txt = BeautifulSoup(fp.read_text(encoding="utf-8",
errors="ignore"), "html.parser").get_text(" ", strip=True)
        if txt:
            out.append((fp.stem, fp.name, txt,
int(datetime.now().timestamp()))))
    return out

# --- Build index (returns ix, idx_dir) ---
def build_index(docs, idx_dir=None):
    schema = Schema(
        doc_id=ID(stored=True, unique=True),
        file_name=ID(stored=True),
        created_at=NUMERIC(stored=True, sortable=True),
        content=TEXT(stored=False, analyzer=StemmingAnalyzer())
    )
    idx_dir = Path(idx_dir or tempfile.mkdtemp(prefix="whoosh_"))
    if idx_dir.exists(): shutil.rmtree(idx_dir, ignore_errors=True)
    idx_dir.mkdir(parents=True, exist_ok=True)
    ix = index.create_in(idx_dir, schema)
    w = ix.writer()
    for doc_id, fname, content, ts in docs:
        w.add_document(doc_id=doc_id, file_name=fname, created_at=ts,
content=content)
    w.commit()
    return ix, idx_dir

# --- Simple searches ---
def search_keywords(ix, text, limit=5, conj="OR"):
    group = OrGroup if conj.upper()=="OR" else AndGroup
    qp = MultifieldParser(["content"], schema=ix.schema, group=group)
    qp.add_plugins([PhrasePlugin(), FuzzyTermPlugin()])
    q = qp.parse(text)
    with ix.searcher() as s:
        return [(h["doc_id"], h["file_name"], float(h.score)) for h
in s.search(q, limit=limit)]

def search_phrase(ix, phrase, limit=5):
    qp = QueryParser("content", ix.schema);
    qp.add_plugins([PhrasePlugin()])
```

```
q = qp.parse(f"\{phrase}\")
with ix.searcher() as s:
    return [(h["doc_id"], h["file_name"], float(h.score)) for h
in s.search(q, limit=limit)]

def search_time_filtered(ix, text, start_ts, end_ts, limit=5):
    qp = QueryParser("content", ix.schema); q = qp.parse(text)
    rng = TermRange("created_at", start_ts, end_ts, startexcl=False,
endexcl=False)
    with ix.searcher() as s:
        return [(h["doc_id"], h["file_name"], float(h.score)) for h
in s.search(q, filter=rng, limit=limit)]

# --- Update/Delete (optional) ---
def update_doc(ix, doc_id, file_name, new_text):
    w = ix.writer(); w.delete_by_term("doc_id", doc_id)
    w.add_document(doc_id=doc_id, file_name=file_name,
created_at=int(datetime.now().timestamp()), content=new_text)
    w.commit()

def delete_doc(ix, doc_id):
    w = ix.writer(); w.delete_by_term("doc_id", doc_id); w.commit()

# --- Example usage ---
if __name__ == "__main__":
    HTML_DIR = Path(r"D:\Sameer\descrip\webinars\vector
database\html_documents")
    docs = load_docs(HTML_DIR)
    ix, idx_dir = build_index(docs)

    print("Keyword (OR):", search_keywords(ix, "AI simulations", 5,
"OR"))
    print("Keyword (AND):", search_keywords(ix, "AI simulations", 5,
"AND"))
    print("Phrase:", search_phrase(ix, "AI in simulations",
5))
    now = int(datetime.now().timestamp())
    print("Time-filtered:", search_time_filtered(ix, "AI",
now-30*24*3600, now, 5))
```

3.3 Building Spare Index - Pinecone

Setup

```
from pathlib import Path
from bs4 import BeautifulSoup
from time import time
import math

from sklearn.feature_extraction.text import TfidfVectorizer
from pinecone import Pinecone, ServerlessSpec

API_KEY = Path("api_key.txt").read_text().strip()
INDEX_NAME = "only-sparse-demo"
HTML_DIR = Path(r"D:\Sameer\descipr\webinars\vector
database\html_documents")
```

Load HTML → texts (+ ids/metadata)

```
files = sorted(HTML_DIR.glob("*.html"))
docs = []
for fp in files:
    txt = BeautifulSoup(fp.read_text(encoding="utf-8",
errors="ignore"),
                        "html.parser").get_text(" ", strip=True)
    if txt:
        docs.append({
            "id": fp.stem,
            "file_name": fp.name,
            "text": txt,
            "created_at": int(time())
        })
```

```
texts = [d["text"] for d in docs]
```

Create Pinecone index (serverless)

```
tfidf = TfidfVectorizer(max_features=100_000, ngram_range=(1, 2))
X = tfidf.fit_transform(texts)          # csr_matrix [n_docs,
vocab_size]

def row_to_sparse_values(csr_row):
    # Convert a 1xV CSR row to Pinecone "sparse_values" {indices,
    values}
    coo = csr_row.tocoo()
    return {
        "indices": coo.col.tolist(),
        "values": [float(v) for v in coo.data.tolist()]
    }

pc = Pinecone(api_key=API_KEY)

if not pc.has_index(INDEX_NAME):
    pc.create_index(
        name=INDEX_NAME,
        vector_type="sparse",           # sparse-only index
        metric="dotproduct",           # required for sparse
        spec=ServerlessSpec(cloud="aws", region="us-east-1")
    )

# Target the index host (recommended)
desc = pc.describe_index(name=INDEX_NAME)
index = pc.Index(host=desc.host)
```

Upsert

```
# Each record has: id, sparse_values, optional metadata
BATCH = 100
payload = []
for i, d in enumerate(docs):
    payload.append({
        "id": d["id"],
        "sparse_values": row_to_sparse_values(X[i]),
        "metadata": {
            "file_name": d["file_name"],
            "created_at": d["created_at"]
        }
    })
    if len(payload) == BATCH:
        index.upsert(vectors=payload)
        payload.clear()
if payload:
    index.upsert(vectors=payload)

print(f"Upserted {len(docs)} sparse vectors into '{INDEX_NAME}'.")
```

Query

```
def lexical_query(query_text: str, top_k: int = 5,
include_text=False):
    q = tfidf.transform([query_text])
    sparse_q = row_to_sparse_values(q)

    res = index.query(
        sparse_vector=sparse_q,          # sparse-only
        top_k=top_k,
        include_values=False,
        include_metadata=True
    )
    out = []
    for m in res.get("matches", []):
        item = {
            "id": m["id"],
```

```
        "score": m["score"],
        "file_name": m["metadata"].get("file_name")
    }
    if include_text:
        item["preview"] = m["metadata"].get("chunk_text") or ""
    out.append(item)
return out

# Try it
results = lexical_query("vector database BM25 TF-IDF hybrid search",
                        top_k=5)
for r in results:
    print(f"{r['id']} | score={r['score']:.4f} | {r['file_name']}")
```

3.4 Advantages of Sparse Indexes

- **Highly efficient for keyword-based search**
Works best when you have exact tokens, IDs, or terms to look up without scanning all data.
- **Space-saving when data is naturally sparse**
Stores only non-zero terms/features, which saves space for text-based or categorical features.
- **Interpretable search results**
Matches are explainable because you know exactly which token/term caused the match.
- **Fast retrieval for inverted lists**
Can jump directly to relevant documents instead of comparing with all vectors.
- **Well-suited for filtering & Boolean queries**
Handles queries like “must include X and must not include Y” easily.

3.5 Disadvantages of Sparse Indexes

1. **Poor performance for semantic similarity**
Doesn't capture meaning beyond exact term matches — synonyms or paraphrases are missed.
2. **Ineffective for noisy or unstructured queries**
Struggles when queries contain typos, abbreviations, or context-dependent terms.

3. Scaling issues with large vocabularies

The inverted index can become huge if the term space is large.

4. Not suitable for multimodal data

Cannot directly work with images, audio, or other non-text embeddings without preprocessing.

5. Limited ranking flexibility

Ranking is usually based on TF-IDF or BM25; lacks rich semantic ranking without hybrid search.

3. Hybrid Indexes — The Best of Both Worlds

So far, we've looked at **dense indexes**, where every vector is a precise, floating-point representation of meaning — think of it as storing the *soul* of your document in a multi-dimensional space.

We've also looked at **sparse indexes**, where we preserve the raw, keyword-level sharpness — like keeping an exact transcript of the words, ready to match if someone says them again.

The trouble?

- Dense indexes understand meaning, but they sometimes hallucinate relevance when the meaning is close but the intent isn't.
- Sparse indexes catch exact words, but fail when someone uses synonyms or slightly different phrasing.

Enter Hybrid Indexes.

A hybrid index is like hiring two detectives: one with a poetic sense for context (dense), other with a sharp memory for details (sparse).

When a query comes in, they both run off, gather their top suspects, and return a merged “most-wanted” list ranked by a combined score.

Why Hybrid Over Just Sparse or Dense?

- **More Robust:** Handles both “tell me exactly” and “tell me roughly” queries in the same breath.
- **Synonym Friendly:** Dense vectors pull in related ideas, sparse keeps the factual anchor.
- **Better Recall + Precision Balance:** Dense widens the net, sparse tightens the catch.

3.1 Creating Hybrid Indexes in Pinecone

In Pinecone, you don't just *flip a switch* to make an index hybrid.

You **create two separate indexes** — one dense, one sparse — and then merge results in your app logic.

The reason? They live in **different vector spaces** — one is floating-point meaning space, the other is integer-token space.

Here's the core change from our earlier sparse index example:

Creating Index

```
from pinecone import Pinecone, ServerlessSpec

pc = Pinecone(api_key=API_KEY)

HYBRID_INDEX = "hybrid-docs" # create a separate index for hybrid
if HYBRID_INDEX not in {getattr(ix, "name", None) or ix.get("name")}
for ix in pc.list_indexes():
    pc.create_index(
        name=HYBRID_INDEX,
        dimension=dense.shape[1], # e.g., 384 for MiniLM
        metric="cosine",
        spec=ServerlessSpec(cloud="aws", region="us-east-1")
    )

hy = pc.Index(HYBRID_INDEX)
```

Upsert

```
vectors_hybrid = []
for i, d in enumerate(docs):
    sp = csr_row_to_pinecone(X[i]) # {"indices": [...],
    "values": [...]}
    vectors_hybrid.append({
        "id": d["id"], # keep your canonical ID
        "values": dense[i].tolist(), # dense embedding
        "sparse_values": sp, # sparse TF-IDF (or
        BM25/SPLADE)
        "metadata": {
            "file_name": d["file_name"],
```

```
        "created_at": d["created_at"],  
        "type": "hybrid"  
    }  
})
```

```
hy.upsert(vectors=vectors_hybrid)
```

3.2 Querying a Hybrid Index

a. Dense-only query (uses only semantic signal)

```
q_dense = emb_model.encode([query], convert_to_numpy=True,  
normalize_embeddings=True)[0].tolist()  
res_dense = hy.query(  
    vector=q_dense,  
    top_k=5,  
    include_values=False,  
    include_metadata=True  
)
```

b. Sparse-only query (uses only lexical signal)

```
q_row = tfidf.transform([query])  
q_sparse = csr_row_to_pinecone(q_row)  
res_sparse = hy.query(  
    sparse_vector=q_sparse,  
    top_k=5,  
    include_values=False,  
    include_metadata=True  
)
```

c. True hybrid (dense + sparse together)

```
alpha = 0.5 # 0..1 (higher favors dense)  
q_dense_w = (np.array(q_dense) * alpha).tolist()  
q_sparse_w = {"indices": q_sparse["indices"], "values": [v * (1 -  
alpha) for v in q_sparse["values"]]}  
  
res_hybrid = hy.query(  
    vector=q_dense_w,
```

```
sparse_vector=q_sparse_w,  
top_k=5,  
include_values=False,  
include_metadata=True  
)
```

How this differs from your sparse-only setup

- **Separate index name** (e.g., **hybrid-docs**) instead of reusing your sparse index.
- **The same record** stores **both values** (dense) and **sparse_values** (lexical).
- **Queries** can be dense-only, sparse-only, or blended (hybrid) **against the same index**.

Retrieval Flow (Dense + Sparse → Hybrid Results)

1. **Dense Search:** Embed query → search in **my-dense-index**.
2. **Sparse Search:** Convert query to sparse vector (BM25/TF-IDF) → search in **my-sparse-index**.
3. **Merge Results:** Assign weights, e.g. **final_score = 0.7*dense_score + 0.3*sparse_score**.
4. **Return Top K** to the user.

Evaluating Retrieval from VectorDB

Now that we know **how to retrieve** the best information — dense, sparse, or even hybrid — the next obvious question is:

How do we know if what we're retrieving is actually the best?

It's like running a library search engine: you've built the shelves, the catalog, and the staff is fetching books quickly... but are they bringing the *right* books? Are they bringing them in the *right order*?

This is where **retrieval evaluation metrics** come in.

The Twist: Order Matters (Sometimes)

You might already know about the classic *Precision* and *Recall* from classification tasks:

- **Precision:** Of all the items I fetched, how many were relevant?
- **Recall:** Of all the relevant items in the whole library, how many did I fetch?

But here's the twist in retrieval systems — *order* often matters.

Imagine you searched for "*best vegetarian restaurants in Mumbai*" and got a list:

```
[1] Some irrelevant pizza place  
[2] A top vegetarian place  
[3] Another vegetarian place  
[4] A steakhouse (oops)
```

Even if the **recall** is high (maybe all vegetarian places are *somewhere* in the list), the fact that the *first result* wasn't vegetarian hurts the user experience.

So, retrieval evaluation metrics are split into two big families:

- **Order-aware metrics** — care about *where* in the list relevant results appear.
- **Order-unaware metrics** — care only about whether relevant results appear in the top K , not where exactly.

1. Order unaware Metrics - Recall@K and Precision@K

Think of a user who only looks at the **first K** results—say $K=5$. We draw a boundary below rank 5 and ask two simple questions:

- **Precision@K:** *Of the K results we showed, what fraction were relevant?*
If top-5 contains 3 relevant docs $\rightarrow$ Precision@5 = $3/5 = 0.60$.
- **Recall@K:** *Of all relevant docs in the whole corpus, how many did we manage to include in the top- K ?*
If there are 8 relevant docs in total and 3 landed in top-5 $\rightarrow$ Recall@5 = $3/8 = 0.375$.

Key point: once you're "inside" the top-K window, **the internal order doesn't matter** to these metrics. A relevant item at rank 1 and rank 5 count equally for Precision@5 and Recall@5—they're both just "in the set."

Use these when you care about **coverage** within the first page and want a simple, readable score.

A tiny example to make it stick

Top-5 results: [A, B, C, D, E]

Relevant set: {B, E, G} (so there are 3 relevant docs overall)

- **Precision@5** = $2/5 = 0.40$ (B and E are inside top-5)
- **Recall@5** = $2/3 \approx 0.67$ (we found 2 of the 3 relevant docs)

2. Order aware Metrics -MRR@K, MAP@K, NDCG@K

Users don't just care whether relevant things are in the top-K—they **care where**. Rank 1 is gold; rank 10 is already late. Order-aware metrics encode that common-sense idea.

- **MRR@K (Mean Reciprocal Rank)**: looks only at the **first** relevant hit. If the first relevant is at rank r , you get $1/r$ (rank-1 $\rightarrow 1.0$, rank-5 $\rightarrow 0.2$). Average over queries. "On average, how far down before the first good answer appears?"
- **MAP@K (Mean Average Precision)**: every time you hit a relevant doc at rank r , compute Precision@ r and average those over **all relevant docs** (missing relevants contribute 0). That's AP for one query; then mean over queries $\rightarrow$ MAP. It rewards **putting many relevants early**, not just the first one.
- **NDCG@K (Normalized Discounted Cumulative Gain)**: give each hit a "gain" (1 for relevant, or graded gains like $3/2/1$), then **discount** by rank (logarithmically). Compare your DCG to the **ideal DCG** (best possible ordering) to get 0–1. This is great when some results are "more relevant" than others.

A tiny example to make it stick

Top-5 results: [A, B, C, D, E]

Relevant set: {B, E, G} (so there are 3 relevant docs overall)

Order-aware now cares **where** B and E landed:

- **MRR@5**: first relevant is **B at rank 2** $\rightarrow 1/2 = 0.5$
- **AP@5** (for this query): precision at ranks where relevant appears
 - at rank 2: $1/2 = 0.5$
 - at rank 5: $2/5 = 0.4$
$$AP = (0.5 + 0.4) / 3 \approx 0.30$$
 (dividing by total #relevant = 3; the missing one contributes 0)
- **NDCG@5**: two gains at ranks 2 and 5, discounted by log; then normalized by the best-possible ordering (which would have put all 3 at the very top). Score ends up < 1 , reflecting the sub-optimal order.

Takeaway: **Order-unaware** tells you “did we get enough in the first page?”; **order-aware** tells you “did we put the best stuff *at the top*?”

```
def precision_at_k(ranked_ids, relevant_ids, k):
    topk = ranked_ids[:k]
    hits = sum(1 for x in topk if x in relevant_ids)
    return hits / max(1, k)

def recall_at_k(ranked_ids, relevant_ids, k):
    if not relevant_ids:
        return 0.0
    topk = ranked_ids[:k]
    hits = sum(1 for x in topk if x in relevant_ids)
    return hits / len(relevant_ids)

def mrr_at_k(ranked_ids, relevant_ids, k):
    topk = ranked_ids[:k]
    for i, x in enumerate(topk, start=1):
        if x in relevant_ids:
            return 1.0 / i
    return 0.0
```

```
def average_precision_at_k(ranked_ids, relevant_ids, k):
    if not relevant_ids:
        return 0.0
    hits, ap = 0, 0.0
    for i, x in enumerate(ranked_ids[:k], start=1):
        if x in relevant_ids:
            hits += 1
            ap += hits / i
    return ap / len(relevant_ids) # unretrieved relevants contribute 0

def dcg_at_k(ranked_ids, gains_dict, k):
    # gains_dict maps doc_id -> gain (e.g., 0/1 or graded 0..3)
    import math
    dcg = 0.0
    for i, x in enumerate(ranked_ids[:k], start=1):
        g = gains_dict.get(x, 0.0)
        dcg += g / math.log2(i + 1)
    return dcg

def ndcg_at_k(ranked_ids, gains_dict, k):
    import math
    dcg = dcg_at_k(ranked_ids, gains_dict, k)
    # build ideal list: sort doc_ids by gain descending
    ideal = sorted(gains_dict.items(), key=lambda kv: kv[1],
reverse=True)
    ideal_ids = [doc_id for doc_id, g in ideal]
    idcg = dcg_at_k(ideal_ids, gains_dict, k)
    return 0.0 if idcg == 0 else dcg / idcg
```

3. System Performance Metrics

3.1 QPS & Latency

In retrieval we often start with *ranking quality*—order-aware and order-unaware metrics such as

nDCG, MRR, precision@k, and recall@k. Those tell us whether the engine retrieves the right things. In production, we must also prove that the system can serve real users in real time. That's where *throughput* and *latency* live. Let's focus on Queries per Second (QPS): what it really is, why it must be paired with latency percentiles, and how to measure it properly on a vector index. We will work on a concrete FAISS HNSW setup, using your already-embedded HTML documents, so students can both run the experiment and interpret the results.

a. What QPS Really Measures

QPS is the number of queries the system completes per wall-clock second over a measurement window. QPS is not an abstract capacity number—it depends on your hardware, index structure, query workload, top-k size, search parameters (e.g., `efSearch` in HNSW), batching, and the degree of concurrency.

b. Why QPS Alone is Not Enough

QPS (Queries per Second) measures *throughput* — how many queries your system can process per second over a given period.

If you only look at QPS, you might think:

“Wow, our vector database handles 200,000 queries/second — it's amazing!”

But here's the trap:

If the **average** query is fast but a small percentage are extremely slow (e.g., take >1 second), those slow queries will still make users wait — and possibly cause **timeouts** in the application.

In real systems, **user experience is dictated by the slowest queries they encounter**, not the average.

c. Latency and Percentiles

Latency is the time between sending a query and receiving a result.

We measure it in **milliseconds** (ms) or seconds, and we rarely stop at the average.

Instead, we look at **percentiles**:

- **p50 latency** — the median: 50% of queries finish in this time or less.
→ Shows the “typical” user experience.
- **p95 latency** — the 95th percentile: 95% of queries finish in this time or less; 5% are slower.
→ Shows the **tail latency** that affects a noticeable minority of requests.

- **p99 latency** — the 99th percentile: 99% of queries finish in this time or less; 1% are slower.
→ Captures the **extreme tail** — the slowest 1% of requests, which often matter for VIP users, system stability, or SLA compliance.

d. Why Tail Latency Matters

Imagine this scenario:

Metric	Value
QPS	150,000
p50 latency	5 ms
p95 latency	50 ms
p99 latency	1,200 ms

On paper, the QPS is fantastic. The **typical** request finishes in 5 ms, so most users are happy. But for the unlucky 1% of queries, results take **over 1 second** — possibly causing:

- **Time-outs** in the front-end or API gateway.
- **UI freezes** in a chat or search interface.
- **Cascading delays** if the slow query is part of a larger workflow (e.g., a RAG pipeline).

This is why a **high QPS with unacceptably high p99** is *not* production-ready — that 1% can still be millions of failed or slow requests per day at scale.

e. QPS and Latency Together Tell the Story

Think of **QPS** as *how much work the system can push through* and **latency** as *how quickly each piece of work finishes*.

- **High QPS + low p99** → Good scaling & stable performance.
- **High QPS + high p99** → Fast for most, but unstable for some (needs investigation).
- **Low QPS + low latency** → Under-utilized; could handle more queries in parallel.
- **Low QPS + high latency** → Performance bottleneck; not ready for production.

Experimental Setup You Will Use

We assume you have already prepared a FAISS HNSW index over MiniLM embeddings of your HTML documents:

- Texts were extracted from HTML via BeautifulSoup.
- Embeddings were created using `sentence-transformers/all-MiniLM-L6-v2`.
- For cosine similarity, we L2-normalize embeddings and use FAISS inner product (cosine on normalized vectors).
- We build an HNSW index with degree `M=32`, construction effort `efConstruction=200`, and query-time knob `efSearch=64` to start.
- We wrap the index in `IndexIDMap2` to preserve stable application-level IDs.

This matters because QPS depends on the index's algorithmic choices. HNSW provides small-world graphs for fast approximate nearest neighbor search; its two main knobs trade accuracy for speed. Larger graphs and higher `efSearch` improve recall but consume more memory and time.

f. How We Will Measure QPS

To measure throughput honestly, we avoid pitfalls that inflate numbers. We do a warm-up to avoid cold-cache artifacts. We encode and normalize all queries once up front so the timing reflects the *search* cost, not the *embedding* cost. Then we run two modes: a single-thread loop (baseline) and a concurrent loop with a thread pool. We also support batching, because FAISS speeds up matrix searches when it sees many query vectors together. Finally, we report QPS and latency percentiles from the same run, so interpretation is straightforward.

Code You Will Run (Single Thread, Concurrency, Percentiles)

Paste the following block directly after your existing setup (where `model`, `hnsw_idmap`, and

your `texts` exist). It produces single-thread and concurrent QPS, as well as p50/p95/p99 latencies. I'll explain the structure right after the code.

```
import time
import numpy as np
from concurrent.futures import ThreadPoolExecutor, as_completed
import faiss

# -----
# 0) Build a realistic query set
# -----
# Use mixed queries to avoid overfitting the cache.
query_texts = [
    "AI in simulations",
    "reinforcement learning in finance",
    "vector database indexing strategies",
    "knowledge graph for RAG",
    "approximate nearest neighbors for retrieval",
    "semantic search over HTML documents",
    "short text queries vs paragraph queries",
    "hybrid sparse-dense retrieval for FAQs",
    "document reranking with cross-encoders",
    "system design for multi-tenant retrieval"
] * 200 # replicate to target a few thousand queries; adjust as
needed

top_k = 10

# -----
# 1) Encode and normalize queries ONCE
# We measure search time, not embedding time.
# -----
q_mat = model.encode(
    query_texts,
    convert_to_numpy=True,
    normalize_embeddings=False
).astype("float32")
faiss.normalize_L2(q_mat)
```

```
print("Encoded queries:", q_mat.shape)

# -----
# 2) Warm-up phase
#   Touch the index so caches/loaders are warm.
# -----
def run_search(qv):
    return hnsw_idmap.search(qv.reshape(1, -1), top_k)

for i in range(min(50, len(q_mat))):
    run_search(q_mat[i])

# -----
# 3) Single-thread measurement
#   Baseline: no concurrency, no batching.
# -----
def measure_single_thread(q_vectors):
    lat_ms = []
    start = time.perf_counter()
    for qv in q_vectors:
        t0 = time.perf_counter()
        _ = hnsw_idmap.search(qv.reshape(1, -1), top_k)
        t1 = time.perf_counter()
        lat_ms.append((t1 - t0) * 1000.0)
    total_time = time.perf_counter() - start
    qps = len(q_vectors) / total_time if total_time > 0 else
float("inf")
    lat_ms = np.array(lat_ms, dtype=np.float64)
    stats = {
        "mode": "single-thread",
        "total_queries": len(q_vectors),
        "total_time_s": total_time,
        "qps": qps,
        "p50_ms": float(np.percentile(lat_ms, 50)),
        "p95_ms": float(np.percentile(lat_ms, 95)),
        "p99_ms": float(np.percentile(lat_ms, 99)),
    }
    return stats
```

```
single_stats = measure_single_thread(q_mat)
print("\n[Single-thread]")
for k, v in single_stats.items():
    print(f"{k}: {v:.3f}" if isinstance(v, float) else f"{k}: {v}")

# -----
# 4) Concurrent measurement
#   Thread pool + optional batching (matrix search).
# -----
def measure_concurrent(q_vectors, max_workers=8, batch_size=8):
    # Split into batches for matrix search
    batches = []
    for i in range(0, len(q_vectors), batch_size):
        batches.append(q_vectors[i:i+batch_size])

    lat_ms = []
    start = time.perf_counter()

    def task(batch):
        t0 = time.perf_counter()
        D, I = hnsw_idmap.search(batch, top_k) # matrix search for
batch
        t1 = time.perf_counter()
        per_query_ms = ((t1 - t0) * 1000.0) / len(batch)
        return per_query_ms, len(batch)

    with ThreadPoolExecutor(max_workers=max_workers) as ex:
        futures = [ex.submit(task, b) for b in batches]
        total_queries = 0
        for fut in as_completed(futures):
            per_q_ms, n = fut.result()
            total_queries += n
            lat_ms.extend([per_q_ms] * n)

    total_time = time.perf_counter() - start
    qps = len(q_vectors) / total_time if total_time > 0 else
float("inf")
    lat_ms = np.array(lat_ms, dtype=np.float64)
```

```
stats = {
    "mode": "concurrent",
    "max_workers": max_workers,
    "batch_size": batch_size,
    "total_queries": len(q_vectors),
    "total_time_s": total_time,
    "qps": qps,
    "p50_ms": float(np.percentile(lat_ms, 50)),
    "p95_ms": float(np.percentile(lat_ms, 95)),
    "p99_ms": float(np.percentile(lat_ms, 99)),
}
return stats

# Try a small grid to illustrate scaling and tail behavior
for workers in [4, 8, 16]:
    for bsz in [1, 8, 32]:
        stats = measure_concurrent(q_mat, max_workers=workers,
batch_size=bsz)
        print(f"\n[Concurrent] workers={workers}, batch_size={bsz}")
        for k, v in stats.items():
            print(f"{k}: {v:.3f}" if isinstance(v, float) else f"{k}:
{v}")
```

g. Understanding the QPS Output

We ran the **same set of 2,000 queries** through our FAISS HNSW index under different concurrency and batching conditions.

For each run, we measured:

1. Single Thread Baseline

- **QPS**: queries processed per second.
- **Latency percentiles**:
 - p50 → median latency (typical query)
 - p95 → tail latency (slowest 5% of queries)
 - p99 → extreme tail (slowest 1%)
- **total_time_s**: wall-clock seconds for the whole run.

```
total_time_s: 0.106
qps: 18,792
p50_ms: 0.051
p95_ms: 0.064
p99_ms: 0.075
```

Interpretation:

With one thread and no batching, the index processes ~18.8k queries/sec, and most queries finish in ~0.05ms.

This is our *baseline*. Every other configuration should be compared against this.

Why the latency is so low:

The dataset is small enough to fit in memory, and FAISS HNSW is optimized for in-memory vector search. CPU cache locality helps.

2. Workers = 4, batch_size = 1

```
qps: 7,284
p50_ms: 0.323
p95_ms: 0.513
p99_ms: 0.974
```

- **Interpretation:**

QPS drops vs. single-thread — unexpected at first glance — because threading overhead and Python GIL contention outweigh parallelism gains for such fast queries.

- **Tail latency** is slightly higher (p99 ~ 1ms) because multiple threads are competing for CPU and memory.

3. Workers = 4, batch_size = 8

```
qps: 552
p50_ms: 0.039
p95_ms: 0.079
p99_ms: 4.520
```


- **Interpretation:**

Here QPS is *much lower* because total run time was artificially inflated — this is likely due to how batches were scheduled across fewer queries per worker in our small dataset. p50 is very low (batch search is fast), but **p99 latency spikes** (~4.5ms) — some batches waited longer in the thread pool queue.

In our QPS experiment, we saw examples:

- **Workers=8, batch_size=32** had very high QPS (~175k) *and* low p99 (~0.03 ms) → stable, production-friendly.
- **Workers=4, batch_size=8** had low QPS *and* a large p99 spike (~4.5 ms) → shows instability in tail queries due to scheduling or queueing.

3.2 Index Build Time, Memory Utilization, Disk I/O (FAISS HNSW)

We've proven the *serving* side (QPS + latency). The next questions are:

1. How long does it take to build the index?
2. How much memory does it occupy?
3. How big is it on disk and how fast can we persist/restore it?

Below, we measure all three for **HNSW** over your MiniLM embeddings.

a. Index Build Time

What it is: Wall-clock time to construct the index (graph creation in HNSW) and add vectors.

Why it matters: Affects data refresh cadence. With frequent updates, slow builds push you toward *incremental* updates; with static or nightly data, full rebuild may be fine.

Key knobs (HNSW): **M** (graph degree), **efConstruction** (build effort). Higher values → better recall, **slower build** and **more memory**.

```
import time
import faiss
import numpy as np

def build_hnsw_idmap_timed(emb_norm: np.ndarray,
                           ids_int64: np.ndarray,
```

```
        M: int = 32,
        efc: int = 200,
        use_ip_metric: bool = True):

    """
    Returns: (idmap_index, stats_dict)
    """

    N, d = emb_norm.shape
    metric = faiss.METRIC_INNER_PRODUCT if use_ip_metric else
faiss.METRIC_L2

    # base HNSW
    try:
        base = faiss.IndexHNSWFlat(d, M, metric)
    except TypeError:
        base = faiss.IndexHNSWFlat(d, M)
    base.hnsw.efConstruction = int(efc)

    # wrap with ID map to enable add_with_ids
    index = faiss.IndexIDMap2(base)

    t0 = time.perf_counter()
    index.add_with_ids(emb_norm, ids_int64)
    t1 = time.perf_counter()

    stats = {
        "N": int(N),
        "dim": int(d),
        "M": int(M),
        "efConstruction": int(efc),
        "build_time_s": t1 - t0,
        "add_rate_vectors_per_s": float(N / (t1 - t0)) if (t1 - t0) >
0 else float("inf"),
        "wrapped_with": "IndexIDMap2"
    }
    return index, stats

# example:
idx_hnsw, build_stats = build_hnsw_idmap_timed(emb_norm, ids_int64,
M=32, efc=200)
```

```
print(build_stats)
```

Output:

```
{'N': 15, 'dim': 384, 'M': 32, 'efConstruction': 200, 'build_time_s':  
0.007361800002399832, 'add_rate_vectors_per_s': 2037.5451649202957, 'wrapped_with':  
'IndexIDMap2'}
```

b. Memory Utilization (RAM)

What it is: Resident memory used by the process after building the index (vectors + graph + overhead).

Why it matters: Determines whether the index fits in your serving box and how much headroom you have for concurrency, reranking, and caches.

Notes: Python's `sys.getsizeof` won't capture FAISS's native allocations; use the process RSS (resident set size).

```
import os, psutil, time

def rss_mb():
    return psutil.Process(os.getpid()).memory_info().rss / (1024**2)

def measure_memory_hnsw_idmap(emb_norm: np.ndarray,
                               ids_int64: np.ndarray,
                               M: int = 32,
                               efc: int = 200,
                               use_ip_metric: bool = True):
    N, d = emb_norm.shape
    metric = faiss.METRIC_INNER_PRODUCT if use_ip_metric else
faiss.METRIC_L2

    try:
        base = faiss.IndexHNSWFlat(d, M, metric)
    except TypeError:
        base = faiss.IndexHNSWFlat(d, M)
```

```
base.hnsw.efConstruction = int(efc)

index = faiss.IndexIDMap2(base)

before = rss_mb()
t0 = time.perf_counter()
index.add_with_ids(emb_norm, ids_int64)
t1 = time.perf_counter()
after = rss_mb()

vector_mb_est = (N * d * 4) / (1024**2) # float32 = 4 bytes
stats = {
    "N": int(N),
    "dim": int(d),
    "M": int(M),
    "efConstruction": int(efc),
    "build_time_s": t1 - t0,
    "rss_before_mb": before,
    "rss_after_mb": after,
    "index_increment_mb": after - before,
    "vector_payload_mb_est": vector_mb_est,
    "graph_overhead_mb_est": (after - before) - vector_mb_est,
    "wrapped_with": "IndexIDMap2"
}

return index, stats

idx_mem, mem_stats = measure_memory_hnsw_idmap(emb_norm, ids_int64,
M=32, efc=200)
print(mem_stats)
```

Output:

```
{'N': 15, 'dim': 384, 'M': 32, 'efConstruction': 200, 'build_time_s':
0.0006387000321410596, 'rss_before_mb': 162.27734375, 'rss_after_mb':
162.328125, 'index_increment_mb': 0.05078125,
'vector_payload_mb_est': 0.02197265625, 'graph_overhead_mb_est':
0.02880859375, 'wrapped_with': 'IndexIDMap2'}
```

c. Disk I/O (save/load)

What it is: Size of the serialized index and the time to write/read it.

Why it matters: Affects cold-start time, container rollouts, and snapshot/backup costs.

```
import os, time

SAVE_PATH = "hnsw_idmap.index"

def write_index_timed(index, path=SAVE_PATH):
    t0 = time.perf_counter()
    faiss.write_index(index, path)
    t1 = time.perf_counter()
    size_mb = os.path.getsize(path) / (1024**2)
    return {
        "file": path,
        "size_mb": size_mb,
        "write_time_s": t1 - t0,
        "write_throughput_mb_s": size_mb / (t1 - t0) if (t1 - t0) > 0
    }
else float("inf"),
}

def read_index_timed(path=SAVE_PATH):
    t0 = time.perf_counter()
    idx = faiss.read_index(path)
    t1 = time.perf_counter()
    size_mb = os.path.getsize(path) / (1024**2)
    return idx, {
        "file": path,
        "size_mb": size_mb,
        "read_time_s": t1 - t0,
        "read_throughput_mb_s": size_mb / (t1 - t0) if (t1 - t0) > 0
    }
else float("inf"),
}
```

```
io_write = write_index_timed(idx_mem, SAVE_PATH)
idx_loaded, io_read = read_index_timed(SAVE_PATH)
print(io_write)
print(io_read)
```

Output:

```
{'file': 'hnsw_idmap.index', 'size_mb': 0.02617931365966797, 'write_time_s':
0.35437269997783005, 'write_throughput_mb_s': 0.07387508592311365}
{'file': 'hnsw_idmap.index', 'size_mb': 0.02617931365966797, 'read_time_s':
0.03555299999425188, 'read_throughput_mb_s': 0.736346121674699}
```

Reranking

So, right now we know how to **retrieve** stuff from a big collection. We've got FAISS, Pinecone, BM25 — all that good stuff — and we can grab the top 100 docs that *seem* relevant.

But here's the catch...

Just because something *looks* close in vector space doesn't mean it's the **best answer** for the user.

Think of it like dating apps:

- Retrieval is when you swipe through and find a bunch of profiles that kinda match what you're looking for.
- Reranking is when you go through those matches more carefully and put the **best ones at the top**.

Why? Because users only look at the first few results. If the top ones are “meh,” they think the whole search sucks.

1. Reranking Explained

Imagine you searched: **"BM25 formula"**.

Your vector retriever might bring up:

- An article about “search algorithms” (good, but not exactly BM25)
- A post about “TF-IDF” (kinda related)
- A random blog on “relevance in search” (too generic)

The real gold — a doc *actually showing* the BM25 equation — might be at rank 12.

Reranking says: *“Wait a sec... exact phrase match? That’s more relevant. Let’s push that up to the top.”*

1.1 Retrieval vs. reranking (same pipeline, different job)

Retrieval is the sprint: be fast, be inclusive, don’t miss potentially relevant items. It optimizes recall and latency. It typically uses pre-built indexes (HNSW, IVF, ScaNN) or lexical posting lists (BM25) to return a few dozen to a few hundred candidates.

Reranking is the interview panel: be thoughtful, be selective, and put the truly best few at the top. It optimizes precision. Because it only handles tens or hundreds of items, it can afford richer scoring—exact term statistics (BM25), cross-encoders that attend over query+document tokens, or even task-aware signals (freshness, authority, click priors). Retrieval narrows, reranking orders.

- **Retrieval** = grab a shortlist from the giant haystack. Be fast, don’t miss stuff.
- **Reranking** = take that shortlist and put the **best** at the very top. Be smart, not just fast.

Retrieval is “who gets invited to the party.” Reranking is “who gets to sit at the VIP table.”

1.2 How we move from retrieval to reranking techniques

With a candidate set in hand, there are three broad families you can teach, from cheapest to most powerful:

- **Lexical rankers** like **BM25**: fast, deterministic, reward exact term overlap with length normalization and IDF weighting. They “snap” semantically close but slightly off results back toward exactness. Great first reranker.
- **Neural bi-encoder rankers**: still independent embeddings, but trained to align query/document vectors specifically for ranking quality. You can reuse ANN search or

cosine sorting here; lighter than cross-encoders.

- **Contextual (cross-encoder) rankers:** read query and candidate together, compute an interaction-aware relevance score. Best precision, highest cost. Modern LLM rerankers are in this bucket.

Start with BM25: it's interpretable, quick to implement, and makes the value of reranking obvious in a single demo.

2. Reranking Example

2.1 BM25 rerank example

Here's how it works in our pipeline:

1. Run your normal vector search to get top 100 docs.
2. Take those 100 and score them again with BM25 (exact term overlap, with length normalization).
3. Sort by that BM25 score.
4. Watch relevant docs jump to the top.

Mini mental picture:

Vector search: "semantic vibes"

BM25: "word-for-word precision"

Together: "relevant vibes *and* right words in the right place."

```
# --- BM25 Rerank on top of your ANN candidates ---
# =====
# BM25 Rerank over ANN (FAISS) - Complete Script
# Paste this AFTER you've created: texts, doc_ids, emb, ids_int64,
model
# And (optionally) built hnsw_idmap (FAISS IndexIDMap2 around HNSW)
# =====

import re
import math
import numpy as np
```



```
# ----- 0) Normalize embeddings once (for cosine via IP) -----
try:
    import faiss
    emb_norm = emb.copy().astype("float32")
    faiss.normalize_L2(emb_norm)
except Exception:
    # fallback if faiss isn't available; we'll normalize with numpy
    in fallback path
    emb_norm = (emb / np.linalg.norm(emb, axis=1,
    keepdims=True)).astype("float32")

# ----- 1) ID → position map (critical if using IndexIDMap2 with
# custom IDs) -----
# ids_int64[k] corresponds to texts[k], doc_ids[k], emb[k]
id_to_pos = {int(ids_int64[k]): k for k in range(len(ids_int64))}

# ----- 2) Tiny tokenizer and BM25 corpus prep -----
def tokenize(s: str):
    return re.findall(r"[a-z0-9]+", s.lower())

corpus_tokens = [tokenize(t) for t in texts]
doc_lens = np.array([len(toks) for toks in corpus_tokens],
dtype=np.float32)
avgdl = float(doc_lens.mean()) if len(doc_lens) else 1.0

# Document frequency for BM25
df = {}
for toks in corpus_tokens:
    for w in set(toks):
        df[w] = df.get(w, 0) + 1

N_docs = len(corpus_tokens)
k1, b = 1.2, 0.75 # classic BM25

def idf(w: str) -> float:
    n = df.get(w, 0)
    # Smooth to avoid negative/inf; classic BM25 variant
    return math.log((N_docs - n + 0.5) / (n + 0.5) + 1.0)
```

```
def bm25_score(query_tokens, doc_idx: int) -> float:
    toks = corpus_tokens[doc_idx]
    if not toks:
        return 0.0
    dl = len(toks)
    norm = k1 * (1 - b + b * dl / avgdl)

    # term freq
    tf = {}
    for w in toks:
        tf[w] = tf.get(w, 0) + 1

    score = 0.0
    for w in query_tokens:
        f = tf.get(w)
        if not f:
            continue
        score += idf(w) * (f * (k1 + 1)) / (f + norm)
    return score

# ----- 3) ANN retrieval (FAISS path if available; cosine fallback
# otherwise) -----
def ann_retrieve(query_text: str, top_k: int = 100):
    q = model.encode([query_text], convert_to_numpy=True,
normalize_embeddings=False).astype("float32")

    # Try FAISS HNSW + IDMap2 first
    try:
        import faiss
        qn = q.copy()
        faiss.normalize_L2(qn) # cosine via IP
        D, I = hnsw_idmap.search(qn, top_k) # returns YOUR custom
IDs

        raw_ids = [int(x) for x in I[0].tolist()]
        positions, scores = [], []
        for rid, sc in zip(raw_ids, D[0].tolist()):
            pos = id_to_pos.get(rid)
            if pos is not None:
```

```
        positions.append(pos)
        scores.append(sc)
    return positions, scores

except Exception:
    # Fallback: cosine against normalized emb_norm (positions
    already aligned)
    qn = (q / np.linalg.norm(q, axis=1,
    keepdims=True)).astype("float32")
    sims = (emb_norm @ qn.T).ravel()
    positions = np.argsort(-sims)[:top_k].tolist()
    scores = [float(sims[i]) for i in positions]
    return positions, scores

# ----- 4) BM25 rerank on the ANN candidate positions -----
def bm25_rerank(query_text: str, cand_positions, top_k_final: int =
10):
    qtoks = tokenize(query_text)
    scored = [(i, bm25_score(qtoks, i)) for i in cand_positions]
    scored.sort(key=lambda x: x[1], reverse=True)
    return scored[:top_k_final] # list[(pos, bm25)]

# ----- 5) Demo: before vs after -----
def demo_bm25_rerank(query: str, ann_k: int = 100, final_k: int = 10,
preview_chars: int = 120):
    cand_positions, vec_scores = ann_retrieve(query, top_k=ann_k)

    print(f"\nQuery: {query}")
    print("\nTop-5 by ANN (semantic):")
    for rank, (i, s) in enumerate(zip(cand_positions[:5],
vec_scores[:5]), start=1):
        snippet = texts[i][:preview_chars].replace("\n", " ")
        print(f"{rank:>2}. id={doc_ids[i]} vec_sim={s:.4f} ::
{snippet}...")

    reranked = bm25_rerank(query, cand_positions,
top_k_final=final_k)

    print("\nTop-10 after BM25 rerank (exact-term boost):")
```

```
for rank, (i, s) in enumerate(reranked, start=1):
    snippet = texts[i][:preview_chars].replace("\n", " ")
    print(f"{rank:>2}. id={doc_ids[i]} bm25={s:.4f} :: {snippet}...")

# ----- 6) Run a couple of test queries -----
demo_bm25_rerank("BM25 formula for lexical reranking in RAG")
demo_bm25_rerank("AI in simulations")
```

2.2 Why it feels similar to Hybrid search

- In a **hybrid search**, you *blend scores* from sparse and dense retrievers before returning results.
- In **ranking**, you *don't blend scores upfront* — you run a second pass, often with a more expensive but accurate model, on already retrieved docs.

So hybrid search = **mix at retrieval stage**,
Reranking = **sort again after retrieval stage**.

2.3. Reranking with Pinecone

```
# ===== Pinecone Rerank using your data (complete) =====
# REQUIRES: pip install -U "pinecone-client>=5.0.0"
# IMPORTANT: use the GRPC import so pc.inference.rerank exists
from pinecone.grpc import PineconeGRPC as Pinecone

# you already have: api_key, model, texts, doc_ids
pc = Pinecone(api_key=api_key)
index = pc.Index("samplevectordb")

# If you didn't store text in Pinecone metadata, map local ids -> text
id_to_text = {str(doc_ids[i]): texts[i] for i in range(len(doc_ids))}
```

```
def pinecone_retrieve(query_text: str, top_k: int = 25):
    # your sentence-transformers model for the query vector
    qv = model.encode([query_text], convert_to_numpy=True,
normalize_embeddings=False).astype("float32")[0].tolist()
    res = index.query(vector=qv, top_k=top_k, include_values=False,
include_metadata=False)
    matches = res.get("matches", []) or res.get("results",
[{}])[0].get("matches", [])
    return [(m["id"], float(m["score"])) for m in matches if "id" in
m]

def build_docs_for_rerank(candidates):
    docs = []
    for _id, _ in candidates:
        text = id_to_text.get(str(_id))
        if text:
            docs.append({"id": str(_id), "text": text})
    return docs

def pinecone_rerank(query_text: str, docs, top_n: int = 5,
model_name: str = "bge-reranker-v2-m3"):
    rr = pc.inference.rerank(
        model=model_name,
        query=query_text,
        documents=docs,          # [{"id": "...", "text": "..."}]
        top_n=top_n,
        return_documents=True
    )
    # rr.data is ranked highest->lowest
    out = []
    for item in rr.data:
        # supports both dict-style and object-style depending on sdk
        doc = item.get("document") if isinstance(item, dict) else
item.document
        score = item.get("score") if isinstance(item, dict) else
item.score
        out.append((doc["id"], float(score), doc["text"]))
    return out
```

```
def demo_pinecone_rerank(query: str, ann_k: int = 25, top_n: int = 5,
    preview_chars: int = 120):
    before = pinecone_retrieve(query, top_k=ann_k)
    docs = build_docs_for_rerank(before)
    if not docs:
        print("No docs to rerank -- check your id_to_text mapping or
store metadata at upsert.")
        return
    after = pinecone_rerank(query, docs, top_n=top_n)

    print(f"\nQuery: {query}")
    print(f"\nTop-{min(5,len(before))} by vector similarity (Stage
1):")
    for rank, (doc_id, score) in enumerate(before[:5], start=1):
        snippet = (id_to_text.get(str(doc_id),
    "")[:preview_chars]).replace("\n", " ")
        print(f"{rank:>2}. id={doc_id} vec_sim={score:.4f} ::
{snippet}...")

    print(f"\nTop-{top_n} after rerank (Stage 2, cross-encoder):")
    for rank, (doc_id, rscore, text) in enumerate(after, start=1):
        snippet = (text or "")[:preview_chars].replace("\n", " ")
        print(f"{rank:>2}. id={doc_id} rerank={rscore:.4f} ::
{snippet}...")

# Example:
# demo_pinecone_rerank("BM25 formula for lexical reranking in RAG",
ann_k=25, top_n=5)
demo_pinecone_rerank("AI in simulations", ann_k=25, top_n=5)
```

Latency Optimization in Retrieval Pipelines

When you run a vector search in Pinecone, you might wonder — “If I search the same thing again, will it have to recompute everything from scratch?” Or, “If I send multiple queries together, do I need to manually optimize the speed?”

1. Caching Frequent Queries

Idea:

If users keep asking the same or very similar questions, we don't need to redo the whole retrieval process every time. Instead, we store (cache) the result once and return it instantly for future identical (or nearly identical) queries.

How it works:

- When a query comes in, check if it's in the **cache store** (Redis, Memcached, or even Pinecone's metadata storage for quick lookups).
- If found, return the stored results immediately without calling the vector database or re-running embeddings + search.
- If not found, process the query normally, but also save the results in the cache for next time.

Example scenario:

Imagine a customer support bot for a bank.

- Users frequently ask: “What is the minimum balance for a savings account?”
- Instead of hitting the vector database and re-ranking results each time, we can serve this from cache in **milliseconds**.

Benefits:

- Reduces load on your vector DB and embedding service.
- Delivers near-instant responses for repetitive queries.
- Frees resources for new, unique queries that actually require computation.

Challenges:

- **Cache invalidation:** When knowledge updates, cached answers might become outdated.

- **Near-duplicate queries:** “Minimum balance savings” vs. “What’s the minimum balance in savings account?” may need fuzzy matching before cache hits.

2. Batch Similarity Searches

Idea:

Instead of sending **one query at a time** to your vector database, send **multiple queries together** as a batch request. This leverages parallel computation in the backend and reduces the number of network round-trips.

How it works:

- Combine multiple embedding vectors into a single request.
- Ask the vector database for top-k results for **all queries in the batch** in one go.
- Post-process results individually.

Example scenario:

Suppose you have a news summarization tool where a user uploads 10 questions at once (e.g., “Give me top 5 articles about AI”, “Top 5 about climate change”, etc.).

- Instead of embedding and searching **each question separately**, you send all embeddings in a single batched request to Pinecone or another DB.
- The DB processes them in parallel internally and returns results faster than 10 separate calls.

Benefits:

- Less **network latency** (fewer HTTP calls).
More efficient CPU/GPU utilization on the backend.
- Particularly useful for **multi-turn conversations** where multiple queries need answers in sequence.

Challenges:

- If queries differ in complexity, some may require more post-processing than others.
- Batching only works well if queries arrive together or within a short time window.

The good news: **Pinecone already handles much of this under the hood** — using techniques like **query caching** and **batch processing** — so you don't always have to manually implement them.

FAISS, on the other hand, is **just a library** (developed by Facebook AI Research) for similarity search. It's extremely fast for what it does, but **it's not a database**.

This means:

1. No Built-in Caching Layer

- FAISS does not store previous query results.
- If you want caching, you have to build it yourself (e.g., store the query → results mapping in Redis or in-memory dictionaries).
- You also have to figure out how to handle "near-duplicate" queries where embeddings are close but not identical.

2. Batch Search is Manual

- FAISS supports searching multiple vectors at once (via `index.search()` on a batch), but you have to manage batching logic, request queues, and concurrency on your own.
- If you want FAISS to handle **millions of queries per second**, you must design your own multi-threading, GPU utilization strategy, or cluster setup.

3. Scaling is DIY

- If your FAISS index grows beyond one machine's memory, you need to manually shard it, load-balance across nodes, and merge results — Pinecone does this automatically.

4. Latency Trade-offs

- Without a managed caching layer, FAISS always performs an index search — even for repeat queries.
- If your embeddings are large and your index is huge, this means **every query re-scans the index** instead of serving from a hot cache.

Conclusion

- **Dense Indexes** – Use embeddings to capture semantic similarity, ideal for concept-level matching rather than exact keywords.

- **Sparse Indexes** – Rely on keyword/token matching (e.g., BM25), great for precise term matches and rare keyword recall.
- **Hybrid Indexes** – Combine dense + sparse approaches to balance semantic relevance and keyword accuracy.
- **Quantization** – Compress embeddings to reduce memory/storage needs, with minimal accuracy loss, enabling larger-scale search.
- **Rerankers** – Apply a secondary model (e.g., BGE Reranker) to refine initial retrieval results for better contextual relevance.
- **Latency Optimization** – Pinecone automatically handles caching of frequent queries, shard distribution, and index balancing to improve retrieval speed.
- **Batch Similarity Searches** – Group multiple queries into a single search request to reduce overhead and network calls, improving throughput.
- **Why FAISS is Different** – While FAISS supports batching, caching frequent queries isn't automatic — it must be implemented manually in the application layer, making dynamic workloads harder to optimize.